

MongoDB: Explicación completa para usar en una API REST

¿Qué es MongoDB?

MongoDB es una base de datos NoSQL orientada a documentos. A diferencia de las bases de datos relacionales (como MySQL o PostgreSQL), MongoDB almacena los datos en documentos JSON (o BSON, su versión binaria) dentro de colecciones.

Es una opción muy popular en aplicaciones modernas gracias a su flexibilidad, escalabilidad y facilidad de integración con JavaScript y Node.js.

¿Para qué sirve?

¿Para qué sirve MongoDB?

- Para guardar información estructurada de forma flexible (sin esquemas fijos).
- Para aplicaciones que cambian rápidamente, donde los datos pueden tener diferentes formas.
- Para sistemas distribuidos y escalables (funciona muy bien en la nube).
- Para manejar grandes volúmenes de datos.

Ejemplos de uso:

- Una app de tareas (to-do list)
- Un sistema de usuarios con roles y permisos variados
- Comentarios en publicaciones (como en redes sociales)

¿Cómo funciona MongoDB?

¿Cómo funciona MongoDB?

- Los datos se guardan en colecciones, que son similares a las tablas en SQL.
- Cada elemento en una colección es un documento, que es como un objeto JSON.
- No necesitas definir un esquema rígido para los documentos (aunque se puede usar con Mongoose para más estructura).

MongoDB: Explicación completa para usar en una API REST

Ejemplo de documento en MongoDB:

```
{
  "_id": "12345",
  "titulo": "Estudiar MongoDB",
  "completada": false,
  "fecha": "2025-04-06T12:00:00Z"
}
```

Usando MongoDB en una API REST

¿Cómo se usa MongoDB en una API REST con Node.js?

La forma más común es usando la librería Mongoose, que permite conectarse y definir modelos para trabajar con MongoDB de forma sencilla.

Pasos básicos:

1. Instalar Mongoose:

```
npm install mongoose
```

2. Conectar a la base de datos:

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost/tareas', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
}).then(() => console.log('Conectado a MongoDB'));
```

3. Crear un modelo:

```
const { Schema, model } = require('mongoose');
const tareaSchema = new Schema({
  titulo: String,
```

MongoDB: Explicación completa para usar en una API REST

completada: Boolean,

fecha: Date

});

const Tarea = model('Tarea', tareaSchema);

4. Usar el modelo en un controlador:

```
exports.crearTarea = async (req, res) => {  
  try {  
    const nuevaTarea = new Tarea(req.body);  
    await nuevaTarea.save();  
    res.status(201).json(nuevaTarea);  
  } catch (error) {  
    res.status(500).json({ mensaje: 'Error al crear la tarea' });  
  }  
};
```

Ventajas y desventajas

Ventajas de MongoDB:

- No necesitas migraciones de esquemas como en SQL.
- Ideal para datos dinámicos y estructuras cambiantes.
- Muy buena integración con el ecosistema JavaScript.
- Fácil de escalar horizontalmente (varios servidores).
- Alta velocidad para lectura/escritura.

Desventajas (a tener en cuenta):

- Menos adecuado si necesitas relaciones complejas entre datos.
- Sin una estructura clara, puede volverse caótico en proyectos grandes sin disciplina.

MongoDB: Explicación completa para usar en una API REST

Resumen

Resumen final:

MongoDB es una base de datos moderna, flexible y potente, ideal para construir APIs REST rápidas y escalables. En el caso de una app de tareas, permite guardar cada tarea como un documento con su título, estado y fecha, y consultarlas o modificarlas fácilmente desde nuestros controladores en Node.js.

¡Es una excelente opción para comenzar con bases de datos en el mundo real!